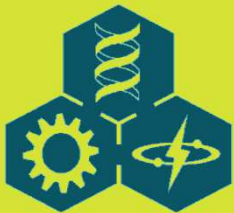
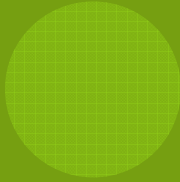


LECTURE 6:

NUMERICAL TECHNIQUES





OUTLINE

6.1 Mistakes Computers Make

6.2 Numerical Integration

6.3 Numerical Instability and Stiff Equations

6.4 Integrating ODEs with Variable Time Steps

6.5 PDEs and the Method of Lines



6.1 MISTAKES COMPUTERS MAKE

◎ Representation of Numbers

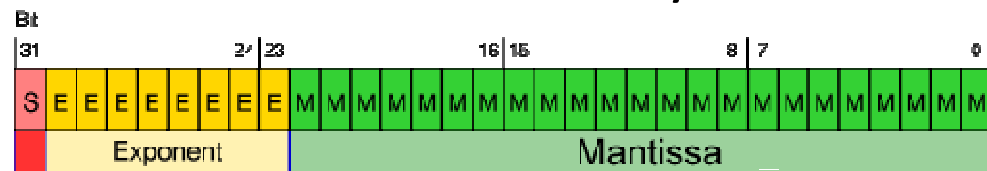
- ◎ In most scientific programming, we are interested in three data types: characters, integers, and real numbers.
- ◎ All data types must be stored using a finite number of bits, and this fact produces the opportunity for error.
- ◎ The value of integer is determined by the *word size* of the computer, which in turn is determined by the size of the databus on the motherboard.
- ◎ More powerful computers have wider databuses.



6.1 MISTAKES COMPUTERS MAKE

◎ Representation of Numbers

- ◎ A floating point number is a real number represented in such a way that the decimal point can float so that a fixed number of significant digits is always represented, no matter how large or small the absolute value of the number.
- ◎ A floating point number is composed of a mantissa and an exponent, either one of which may be positive or negative.
- ◎ Exponents are integers, while the mantissa is interpreted as a real number scaled by the exponent.



6.1 MISTAKES COMPUTERS MAKE

◎ Representation of Numbers

- ◎ The number of bits used for the mantissa represents the precision (number of significant digits) of the number.
- ◎ Single Precision vs. Double Precision

Table 6.1: Format parameters for single and double precision numbers in the IEEE 754 standard for floating point numbers. Shown are the number of bits used for mantissa and exponents; the approximate number of decimal significant digits, and the maximum and minimum numbers.

	Mantissa	Exponent	Sig. Digits	Max	Min
Single	23	8	9	3.403×10^{38}	1.175×10^{-38}
Double	52	11	15	1.798×10^{308}	2.225×10^{-308}

- ◎ If the exponent is negative and the operation on the exponent causes an *overflow* in the exponent bit pattern, the condition is called floating point *underflow*, since the operation attempted to create a number smaller than that which could be represented.



6.1 MISTAKES COMPUTERS MAKE

- ◎ Round-Off, Truncation, and Propagation Errors
 - ◎ Storage limitations in the mantissa produce *overflow* or *underflow* and these become *round-off errors*.
 - ◎ Modern floating point chips that implement the IEEE 754 provide the programmer the ability to determine what method to use. The choices include *always round up*, *always round down*, or *round to nearest*. The most accurate (and default) method is to round to nearest.



6.1 MISTAKES COMPUTERS MAKE

- ◎ Round-Off, Truncation, and Propagation Errors
 - ◎ Round-off issues can have important implications for basic scientific programming.
 - ◎ Example:

_____ **Bad** _____

```
mean=0.0;  
for (i: 1 → N) {  
    mean = mean + A[i]  
}  
mean = mean/N
```

_____ **Good** _____

```
gmean=0.0;  
for (i: 1 → N) {  
    gmean = gmean + (A[i]-gmean)/i  
}
```



6.1 MISTAKES COMPUTERS MAKE

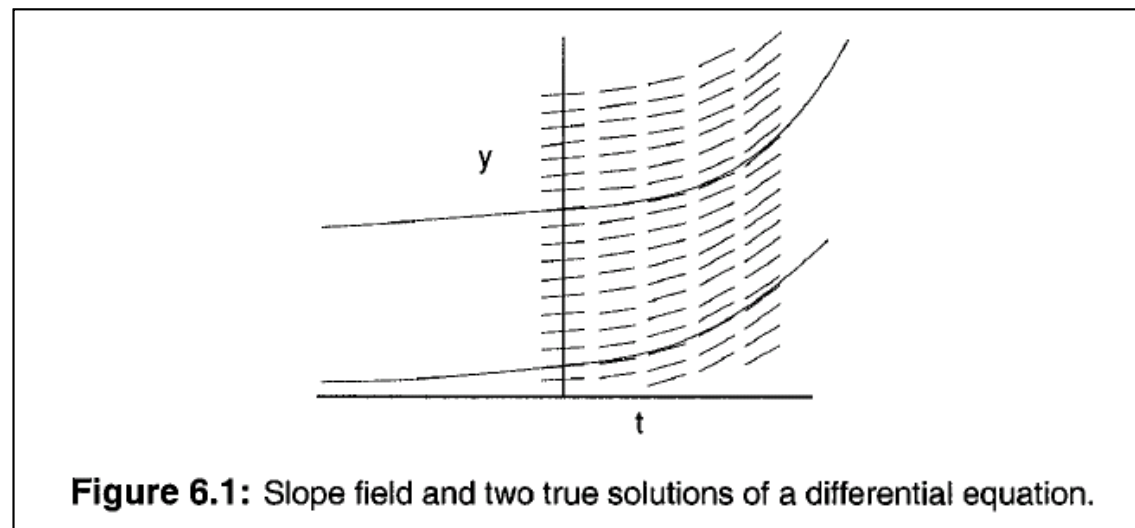
- ◎ Round-Off, Truncation, and Propagation Errors
 - ◎ Two other kinds of errors occur depending on the operations used in the algorithm: *truncation errors* and *propagation errors*.
 - ◎ *Truncation errors* occur because the algorithm approximates a function as an infinite series truncated after the first n terms.
 - ◎ *Propagation errors* are errors made at every stage of an iterative algorithm and that accumulate over the entire solution.
 - ◎ In an iterative procedure, these sources produce two types of error: *local error* (at every solution step) and *global error* (deviation from the true solution).



6.2 NUMERICAL INTEGRATION

◎ Slope Fields

- ◎ The solutions of an ordinary differential equation can be plotted in a two-dimensional space in which the y -axis is the dependent variable and the x -axis is time (t).



6.2 NUMERICAL INTEGRATION

◎ Slope Fields

- ◎ Taking the derivative of the solution function (generally unknown) at a point on the time axis will give the numerical values of the original differential equation for the particular (y, t) pair. If we calculate the derivative at many of these pairs, we will produce a slope field.
- ◎ The problem in numerical approximation of the true solution is to find the subset of slopes in the slope field that corresponds to the true solution.



6.2 NUMERICAL INTEGRATION

⊙ Euler's Method

- ⊙ The strategy of numerical integration is to move from the initially correct slope in the slope field to the next correct slope, from there to the next correct slope, and so on.
- ⊙ The Euler method is the simplest, most straightforward approximation.

$$y_{t+\Delta t} \cong y_t + \Delta t \cdot f(y_t, t) \quad (6.1)$$



6.2 NUMERICAL INTEGRATION

🎯 Euler's Method

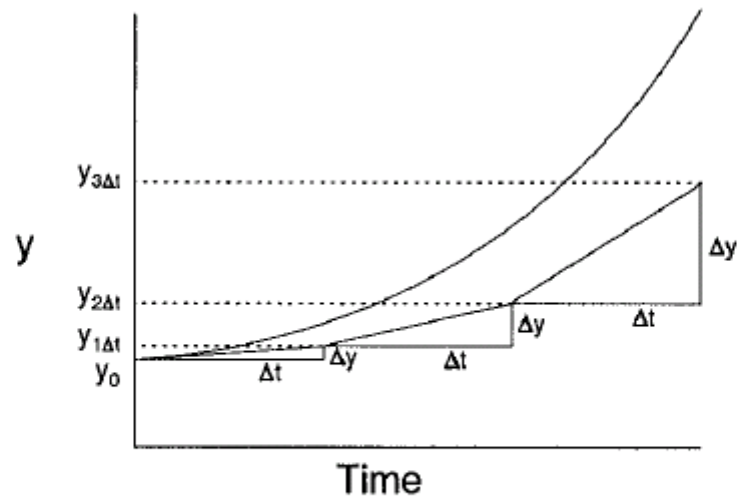


Figure 6.2: A series of Euler approximations (straight lines) to a true solution (curved line) over Δt solution intervals.



6.2 NUMERICAL INTEGRATION

◎ Euler's Method

- ◎ Typically, we must solve several differential equations simultaneously and these equations are a system in the sense that their derivatives are functions of the other state variables. For example, a model of predator and prey populations is

$$\frac{dV}{dt} = rV - bVP \quad \frac{dP}{dt} = bcVP - dP \quad (6.2)$$

- ◎ In the Euler method, these continuous equations are replaced by the approximations:

$$\begin{aligned} V_{t+\Delta t} &= V_t + [rV_t - bV_tP_t]\Delta t \\ P_{t+\Delta t} &= P_t + [bcV_tP_t - dP_t]\Delta t \end{aligned} \quad (6.3)$$

- ◎ We should always first calculate the rates (derivatives), then update the states.



6.2 NUMERICAL INTEGRATION

◎ Runge-Kutta Basics

- ◎ The Runge-Kutta (RK) method has many advantages. It is easy to code; its numerical behavior is less sensitive to the size of Δt than the Euler method. In addition, it is remarkably efficient: a large Δt provides accurate solutions.
- ◎ The Runge-Kutta method uses several estimates of the slope of the function.
- ◎ The Runge-Kutta is actually a family of algorithms in which the members are distinguished by the number of slope (derivatives) calculations performed and weights given to those slopes.



6.2 NUMERICAL INTEGRATION

Runge-Kutta Basics

- When the number of derivatives computed is two, we have the second-order Runge-Kutta (RK-2, also known as *the mid-point method*).

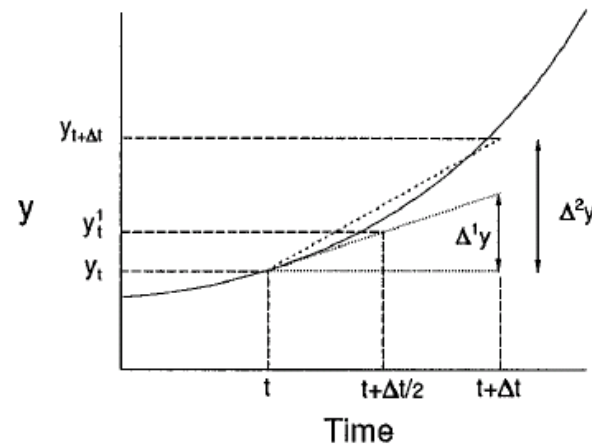


Figure 6.3: Second-order Runge–Kutta integration. Δ^2y is the second estimate of the rate of change based on the midpoint value y_t^2 ; $y_{t+\Delta t}$ is obtained by weighting Δ^2y and adding to y_t .



6.2 NUMERICAL INTEGRATION

◎ Runge-Kutta Basics

◎ The algorithm of RK-2

1. Calculate derivative 1 using current solution and then first tentative solution

$$\Delta^1 y = f(y_t, t) \Delta t$$

$$y^1 = y_t + \Delta^1 y / 2 \quad (\text{Tentative step based on } 1/2 \text{ time step})$$

2. Calculate derivative 2 using tentative solution 1.

$$\Delta^2 y = f(y^1, t + \Delta t / 2) \Delta t$$

(No further tentative steps needed)

3. Calculate new value for y by combining the previous $\Delta^i y$ with different weights.

$$y_{t+\Delta t} = y_t + (0 \cdot \Delta^1 y) + (1 \cdot \Delta^2 y)$$



6.2 NUMERICAL INTEGRATION

◎ Runge-Kutta Basics

◎ An example of RK-2

The numerical calculations for one time step of the RK-2 on the equation $dy/dt = ay$,

with $y(0) = 10.0$, $a = 0.5$, $\Delta t = 1.0$ are :

$$\Delta^1 y_t = (0.5)(10)(1.0) = 5.0$$

$$y_t^2 = 10 + 5.0 / 2 = 12.5$$

$$\Delta^2 y_t = (0.5)(12.5)(1.0) = 6.25$$

$$\Delta^* y_t = 5.0 \cdot 0 + 6.25 \cdot 1 = 6.25 \quad \leftarrow \text{Weighted } \Delta^i y$$

$$y_{t+\Delta t} = 10.0 + 6.25 = 16.25$$

Compare this estimate with the true solution :

$$y_{1.0} = 16.4872$$



6.2 NUMERICAL INTEGRATION

◎ Runge-Kutta Basics

◎ The algorithm of RK-4

1. Calculate derivative 1 using current solution and then first tentative solution

$$\Delta^1 y = f(y_t, t) \Delta t$$

$$y^1 = y_t + \Delta^1 y / 2 \quad (\text{Tentative step based on 1/2 time step})$$

2. Calculate derivative 2 using tentative solution 1 and then second tentative solution.

$$\Delta^2 y = f(y^1, t + \Delta t / 2) \Delta t$$

$$y^2 = y_t + \Delta^2 y / 2 \quad (\text{Tentative step based on 1/2 time step.})$$

3. Calculate derivative 3 using tentative solution 2 and then third tentative solution.

$$\Delta^3 y = f(y^2, t + \Delta t / 2) \Delta t$$

$$y^3 = y_t + \Delta^3 y \quad (\text{Tentative step based on 1 whole time step.})$$



6.2 NUMERICAL INTEGRATION

◎ Runge-Kutta Basics

◎ The algorithm of RK-4

4. Calculate derivative 4 using tentative solution 3

$$\Delta^4 y = f(y^3, t + \Delta t) \Delta t$$

Last tentative solution not needed.

5. Calculate new value for y by combining the previous $\Delta^i y$ with different weights.

$$\Delta^* y_t = \frac{1}{6}(\Delta^1 y + 2(\Delta^2 y + \Delta^3 y) + \Delta^4 y)$$

$$y_{t+\Delta t} = y_t + \Delta^* y_t$$



6.2 NUMERICAL INTEGRATION

☉ Runge-Kutta Basics

Table 6.2: Comparison of Runge–Kutta and Euler methods solving $dy/dt = ay$, $a = 0.5$, $\Delta t = 1.0, 0.5, 0.25$.

Time	Euler $\Delta t = 1.0$	Euler $\Delta t = 0.5$	Euler $\Delta t = 0.25$	RK-2 $\Delta t = 1.0$	RK-4 $\Delta t = 1.0$	True
0.0	10.0	10.0	10.0	10.0	10.0	10.0
1.0	15.0	15.625	16.0181	16.2500	16.4844	16.4872
2.0	22.5	24.400	25.6579	25.3900	27.1735	27.1828

1. All methods become less accurate over time.
2. The Euler method becomes more accurate as Δt decreases.
3. The Euler method is less accurate than the Runge-Kutta method even when the methods use the same number of derivative calculations [e.g., Euler ($\Delta t = 0.5$) versus RK-2, and Euler ($\Delta t = 0.25$) vs RK-4].
4. RK-4 is remarkably accurate for this simple ODE.



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

- Desirable integration methods are those that reduce the errors more effectively at large step sizes. RK is generally more effective for many more problems than Euler, but RK fails for certain equations.
- A prime example of these are stiff equations. Stiffness can arise when the equations use several, very different time scales.
- Different time scales in equations often cause the solution algorithm to add very large numbers to very small numbers. This is a situation that produces large round-off and truncation errors.



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

- ◎ Some examples of systems whose differential equations may be stiff are:
 1. *Algal Nutrient Uptake and Cellular Division*: Nutrient uptake is a rapid process that occurs over microseconds; cell division requires several hours (Abbott 1990).
 2. *Photosynthesis and Enzymatic Reactions*: Oscillating light levels will produce a rapid change in enzyme kinetic parameters but a relatively slow change in photosynthesis at the leaf level (Gross 1982).
 3. *Rotating Rocket Orbiting Earth*: The rocket rotation is fast compared to the orbiting time (Rice 1983).
 4. *Refinery Control*: Chemical reactions occur rapidly compared to the temperature response of the large vats (Rice 1983).



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

- ◎ There are two broad approaches to solving this problem of multiple time scales:
 1. The first method is most applicable to computer simulation in which we create *submodels* that correspond to the subsystems having different time scales.
 2. The second approach comes from physics and does not attempt to identify and model specific subprocesses that account for the existence of the time scales. Rather than modeling these as separate subsystems, a numerical approach is to find a better method of integrating the equations.



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

⊙ Examples of Stiff Equations:

- ⊙ The problem of stiff equations arises simply because the parameters in the system of ODES vary over several magnitudes.

$$\frac{du}{dt} = 998u + 1998v \quad \frac{dv}{dt} = -999u - 1999v \quad (6.4)$$

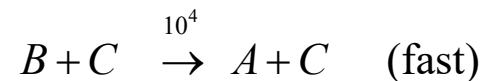
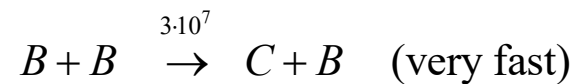
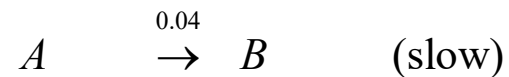
- ⊙ Mathematically, stiff equations are a practical problem in linear systems such as this when all the eigenvalues are negative and the largest eigenvalue is very much larger (at least 10 times) than the smallest eigenvalue.
- ⊙ There are two possible solutions: (1) decrease the step size appropriate to the fastest time scale, and (2) use a different numerical method.



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

Examples of Stiff Equations:

Robertson's Chemical Reaction Systems



which lead to the equations

$$A: \quad \frac{dy_1}{dt} = -0.04y_1 + 10^4 y_2 y_3 \quad y_1(0) = 1$$

$$B: \quad \frac{dy_2}{dt} = 0.04y_1 - 10^4 y_2 y_3 - 3 \cdot 10^7 y_2^2 \quad y_2(0) = 0$$

$$C: \quad \frac{dy_3}{dt} = 3 \cdot 10^7 y_2^2 \quad y_3(0) = 0$$



6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

Examples of Stiff Equations:

Robertson's Chemical Reaction Systems

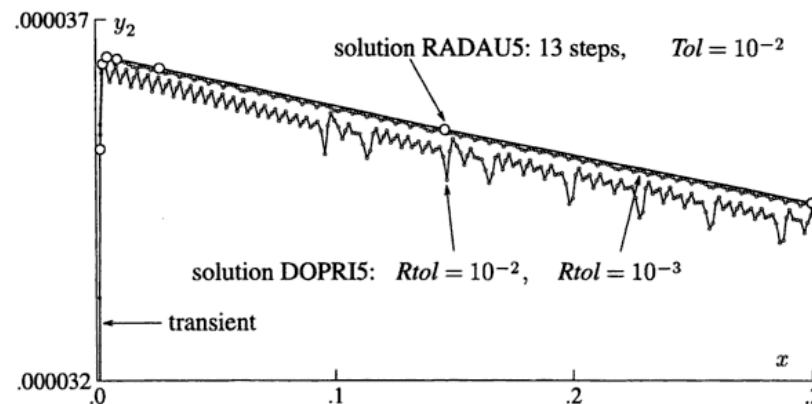


Fig. 1.3. Numerical solution for problem (1.4) with DOPRI5 and RADAU5

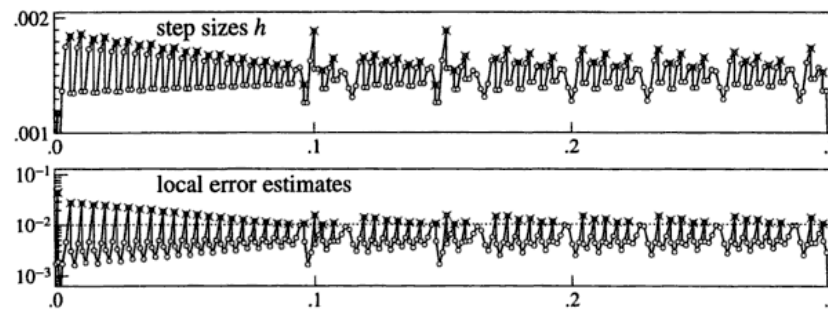


Fig. 1.4. Step sizes and local error estimates of DOPRI5, $Tol = 10^{-2}$



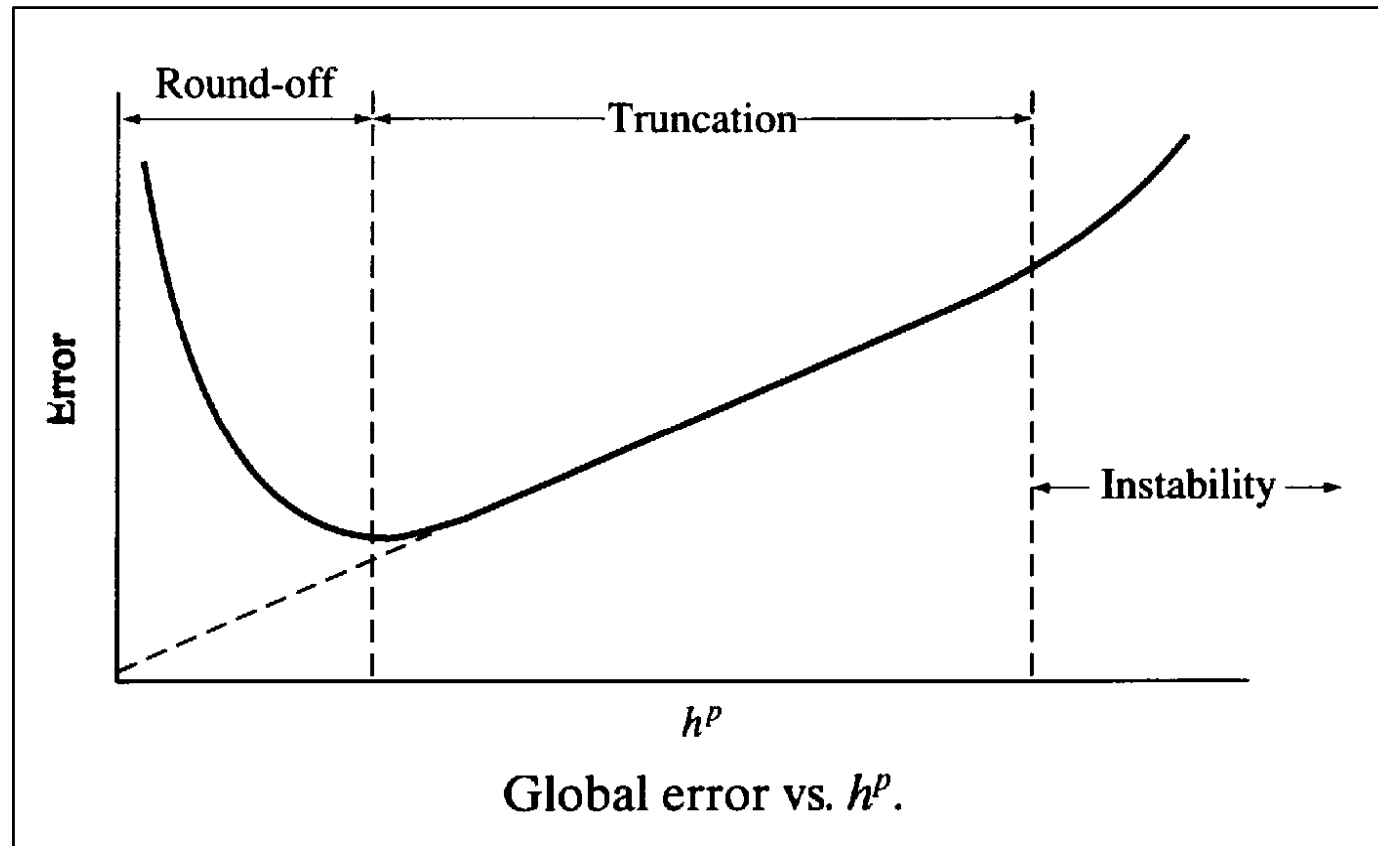
6.3 NUMERICAL INSTABILITY AND STIFF EQUATION

- ◎ In conclusion, time scales and stiff equations are a potential problem because biological dynamics occur over many different time scales.
- ◎ It is advisable, when studying equations with which one does not have much previous experience, to monitor the net rates of changes of each state variable. The relative net rates of change should stay within reasonable bounds.
- ◎ As a very crude check, if $(1/x_i)(dx_i/dt) > 0.2$ in any time step, then you should consider reducing the time step or using methods developed for stiff equations.
- ◎ At the least, during preliminary modeling stages, the modeler should vary the simulation time step over a wide range to determine the presence of spurious behavior.



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

Effect of the Step Size on the Global Errors



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

- ◎ The simplest approach to optimizing the time step for any integration method is to calculate, at every iteration, the estimate for the next value using the current time step and an estimate using a smaller time step.
- ◎ If these differ by an unacceptable amount, then the truncation error is too great and a smaller step size is needed. This test is repeated as many times as necessary within the current time step until the error criterion is satisfied.



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

◎ For the Euler method, the calculations are:

$$y_{t+\Delta t} = y_t + \Delta t f(y_t, t) \quad \leftarrow \text{full step}$$

$$y_{t+\Delta t/2}^* = y_t + (\Delta t / 2) f(y_t, t) \quad \leftarrow \text{midpoint value}$$

$$y_{t+\Delta t}^* = y_{t+\Delta t/2}^* + (\Delta t / 2) f(y_{t+\Delta t/2}^*, t) \quad \leftarrow \text{two half steps}$$

The absolute (global) error estimate is

$$E_{\Delta t} = |y_{t+\Delta t} - y_{t+\Delta t}^*|$$

and the error relative to the current magnitude of the state variable is

$$e_{\Delta t} = \frac{E_{\Delta t}}{y_t}$$



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

- Given the calculated $e_{\Delta t}$, we can calculate another $\Delta't$ which is the time step needed to exactly produce the target or desired error.
- The error estimates are proportional to $(\Delta t)^2$. We use this fact to note that if $e_{\Delta t} \propto (\Delta t)^2$, then there is a target error proportional to some other time step: $e'_{\Delta t} \propto (\Delta't)^2$. Using these two proportionalities, we have:

$$\Delta't = \Delta t \left(\frac{e'_{\Delta t}}{e_{\Delta t}} \right)^{1/2}$$



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

- ◎ This variable time step approach also applies to RK-4, but each of the four steps must be performed for both the full time step and the two half time steps.
- ◎ Therefore, in the step-doubling method for RK-4, we must calculate the derivative 11 times, as compared to 4 for the nonvariable method.
- ◎ The Runge-Kutta-Fehlberg (RKF or RKF45) method which is an alternative that also uses an estimate of the truncation error to determine the best time step. This method is a fifth-order RK method that requires six calculations of the derivatives.
- ◎ The major feature of the RKF algorithm is that it gives an error estimate using only six evaluations of the derivatives, rather than the 11 needed for the time step varying method described above.



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

◎ MATLAB functions

Syntax

`[T,Y] = solver(odefun,tspan,y0)`

`[T,Y] = solver(odefun,tspan,y0,options)`

`[T,Y,TE,YE,IE] = solver(odefun,tspan,y0,options)`

`sol = solver(odefun,[t0 tf],y0...)`

where *solver* is one of `ode45`, `ode23`, `ode113`, `ode15s`, `ode23s`, `ode23t`, or `ode23tb`.

Solver	Problem Type	Order of Accuracy	When to Use
ode45	Nonstiff	Medium	Most of the time. This should be the first solver you try.
ode23	Nonstiff	Low	For problems with crude error tolerances or for solving moderately stiff problems.
ode113	Nonstiff	Low to high	For problems with stringent error tolerances or for solving computationally intensive problems.
ode15s	Stiff	Low to medium	If ode45 is slow because the problem is stiff.
ode23s	Stiff	Low	If using crude error tolerances to solve stiff systems and the mass matrix is constant.
ode23t	Moderately Stiff	Low	For moderately stiff problems if you need a solution without numerical damping.
ode23tb	Stiff	Low	If using crude error tolerances to solve stiff systems.



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

◎ MATLAB functions

% Main program - ode45demo

function ode45demo

options = odeset('RelTol', 1e-4, 'AbsTol',[1e-4 1e-5]);

[T,Y] = ode45(@model1, [0 600], [100 200], options);

plot(T,Y(:,1),'-',T,Y(:,2),'-.');

% Prey Predator Model called by ode45

function dy = model1(t,y)

k1 = 0.05;

k2 = 0.10;

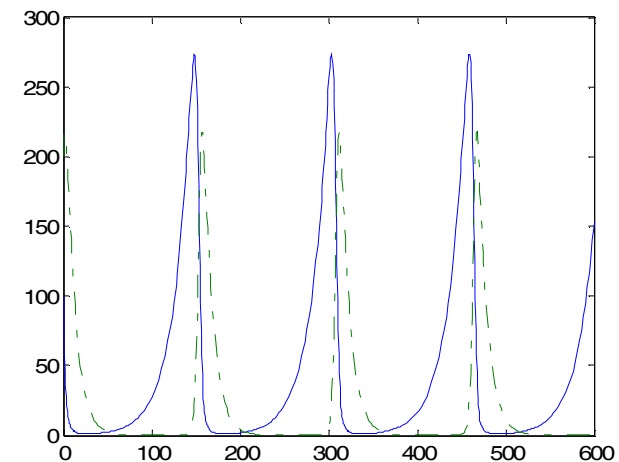
k3 = 0.002;

k4 = 0.002;

dy = zeros(2,1); % a column vector

dy(1) = (k1-k3*y(2))*y(1);

dy(2) = (k4*y(1)-k2)*y(2);



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

◎ Python Packages

Package	Key Feature
SciPy	General-purpose ODE solver (stiff & non-stiff)
SymPy	Exact solutions for ODEs
JAX	GPU-accelerated, differentiable ODEs
TensorFlow Probability	ODE solvers in ML models
TorchDiffEq	Neural ODEs in PyTorch
DifferentialEquations.jl	Fastest solvers via Julia



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

◎ Python SciPy Packages

Functions	When to Use
<code>solve_ivp</code> (fun, t_span, y0[, method, t_eval, ...])	Solve an initial value problem for a system of ODEs.
<code>RK23</code> (fun, t0, y0, t_bound[, max_step, rtol, ...])	Explicit Runge-Kutta method of order 3(2).
<code>RK45</code> (fun, t0, y0, t_bound[, max_step, rtol, ...])	Explicit Runge-Kutta method of order 5(4).
<code>DOP853</code> (fun, t0, y0, t_bound[, max_step, ...])	Explicit Runge-Kutta method of order 8.
<code>Radau</code> (fun, t0, y0, t_bound[, max_step, ...])	Implicit Runge-Kutta method of Radau IIA family of order 5.
<code>BDF</code> (fun, t0, y0, t_bound[, max_step, rtol, ...])	Implicit method based on backward-differentiation formulas.
<code>LSODA</code> (fun, t0, y0, t_bound[, first_step, ...])	Adams/BDF method with automatic stiffness detection and switching.
<code>OdeSolver</code> (fun, t0, y0, t_bound, vectorized)	Base class for ODE solvers.
<code>DenseOutput</code> (t_old, t)	Base class for local interpolant over step made by an ODE solver.
<code>OdeSolution</code> (ts, interpolants[, alt_segment])	Continuous ODE solution.



6.4 INTEGRATING ODEs WITH VARIABLE TIME STEPS

Python SciPy functions

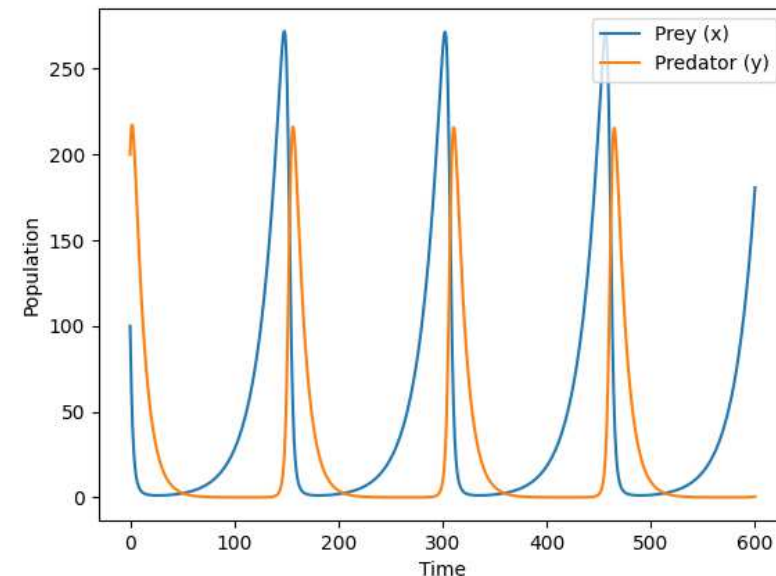
```
import numpy as np
from scipy.integrate import solve_ivp, odeint
import matplotlib.pyplot as plt

def lotka_volterra(t, z, k1, k2, k3, k4):
    x, y = z # Unpack variables
    dxdt = k1 * x - k3 * x * y
    dydt = k4 * x * y - k2 * y
    return [dxdt, dydt]

# Parameters
k1, k2, k3, k4 = 0.05, 0.10, 0.002, 0.002
# Initial conditions: x0 = 100 (prey), y0 = 200 (predators)
y0 = [100, 200]
# Time span
t_span = (0, 600)
t_eval = np.linspace(0, 600, 1000)

# Solve
sol = solve_ivp(lotka_volterra, t_span, y0, args=(k1, k2, k3, k4), t_eval=t_eval)

# Plot
plt.plot(sol.t, sol.y[0], label="Prey (x)")
plt.plot(sol.t, sol.y[1], label="Predator (y)")
plt.xlabel("Time")
plt.ylabel("Population")
plt.legend()
plt.show()
```



6.5 PDEs AND THE METHOD OF LINES

⊙ Discretization

- ⊙ To obtain an approximate, numerical solution to the continuous equations, we discretize continuous space into a large, but finite, number of grid points.
- ⊙ A common approximation for advection at node i is the midpoint of the slope defined by the neighboring nodes :

$$U_x \left(\frac{\partial q}{\partial x} \right)_i \approx U_x \left(\frac{q_{i+1} - q_{i-1}}{2\Delta x} \right)_i \quad (6.5)$$

- ⊙ Likewise, a reasonable approximation for the second-order diffusion process is

$$D \left(\frac{\partial^2 q}{\partial x^2} \right)_i \approx D \left(\frac{(q_{i+1} - q_i) - (q_i - q_{i-1}))}{\Delta x^2} \right)_i \approx D \left(\frac{q_{i+1} - 2q_i + q_{i-1}}{\Delta x^2} \right)_i \quad (6.6)$$



6.5 PDEs AND THE METHOD OF LINES

⊙ Discretization

- ⊙ In typical mass transport models, the processes that move mass (or energy and momentum) are additive in two or three dimensions.
- ⊙ The above method of discretization is called central differencing because the scheme is centered around the node currently being evaluated (i in Eqs. 6.5 and 6.6).
- ⊙ There are two broad families of discretization methods: If time and space are both discretized, the classical finite difference or finite element methods based on solving a set of algebraic equations are used.
- ⊙ If time is not discretized, but space is, we use the *method of lines*.



6.5 PDEs AND THE METHOD OF LINES

Method of Lines and ODEs

- Consider the flow of a contaminant in a river (p) with advection, molecular diffusion, and bioaccumulation in biotic components (b). A plausible model might be:

$$\frac{\partial p}{\partial t} = -U_x \frac{\partial p}{\partial x} + D \frac{\partial^2 p}{\partial x^2} - kb \left(1 - \frac{b}{B}\right) \quad (6.7)$$

$$\frac{\partial b}{\partial t} = kb \left(1 - \frac{b}{B}\right) - v_x \frac{\partial b}{\partial x}$$

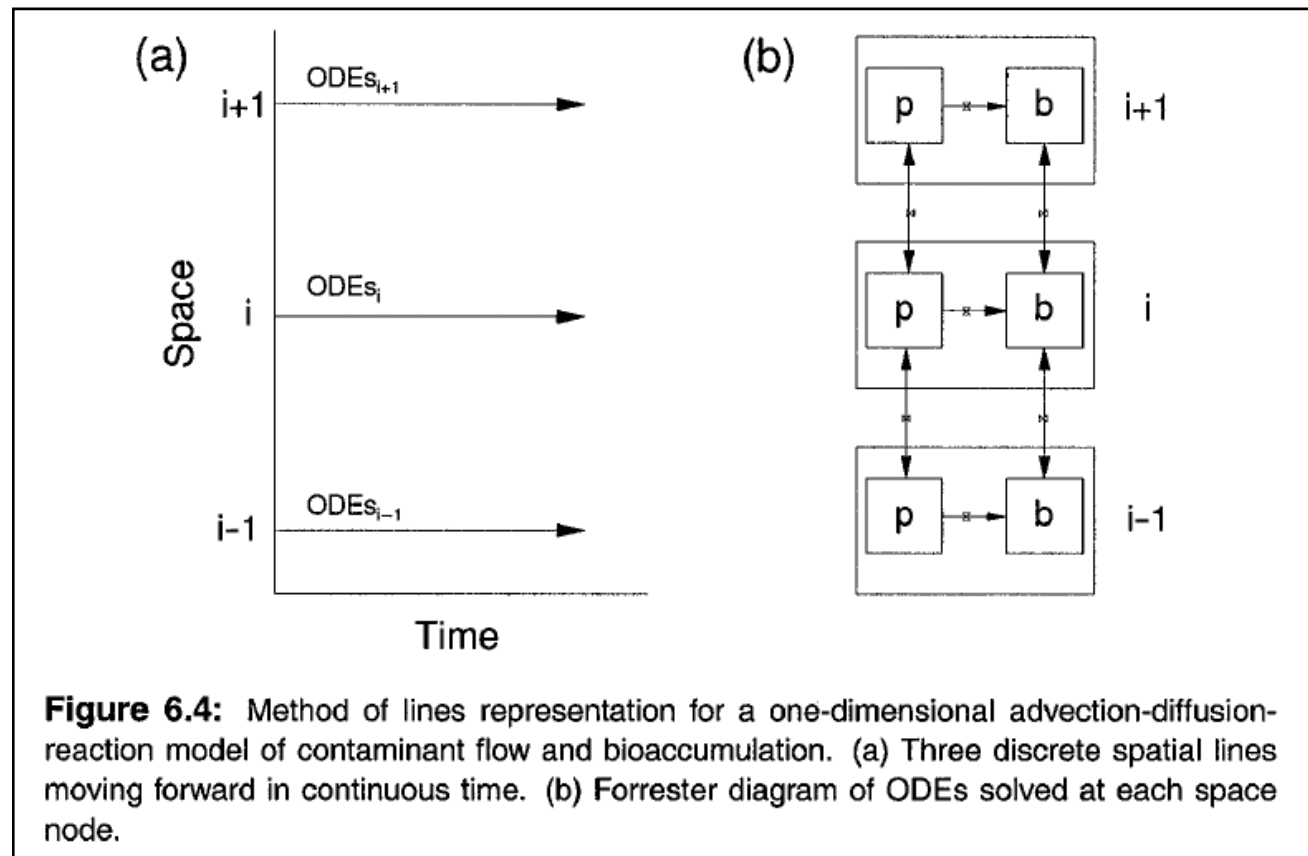
where the velocity in the x direction is U_x , D is diffusivity, and contaminant uptake (k) by biota decreases as the amount of the biota (b) increases to a maximum biomass.

- Equations such as Eqs. 6.7 are called *reaction-diffusion equations*.



6.5 PDEs AND THE METHOD OF LINES

Method of Lines and ODEs



6.5 PDEs AND THE METHOD OF LINES

Method of Lines and ODEs

- Using i to index the nodes, the ODEs that must be solved at each node are

$$\frac{\partial p_i}{\partial t} = -U_x \frac{p_{i+1} - p_{i-1}}{2\Delta x} + D \frac{p_{i+1} - 2p_i + p_{i-1}}{\Delta x^2} - kb \left(1 - \frac{b}{B}\right) \quad (6.8)$$

$$\frac{\partial b_i}{\partial t} = kb \left(1 - \frac{b}{B}\right) - v_x \frac{b_{i+1} - b_{i-1}}{2\Delta x}$$

- The time step and the spatial grid size are interrelated. If the grid size is too large, we may not correctly represent the dynamics at any node. If the grid size is too small, we will perform unnecessary calculations.



6.5 PDEs AND THE METHOD OF LINES

◎ Boundary Conditions

- ◎ Equations 6.8 will work well for grid nodes that are on the interior of the space being simulated. We must treat the boundary nodes differently because they do not have all the neighbors required by the equations.
- ◎ Four possible cases of boundary conditions:
 1. The boundary is a true boundary (edges or corners).
 2. Boundary of a grid embedded in a larger grid.
 3. The grid may be embedded in a virtual grid in which the boundary nodes are "fictitious".
 4. The topology need not conform to physical space.
- ◎ Two basic approaches to define edge nodes:
 1. Force the values of the boundary nodes to specific values (e.g., 0.0, but which may vary in time),
 2. Set the fluxes into or out of the boundary nodes to some specific magnitude (which may also vary in time).



6.5 PDEs AND THE METHOD OF LINES

◎ Example Model of Butterfly Wing Patterns

$$\frac{\partial A}{\partial t} = -k_1 A$$

$$\frac{\partial M_1}{\partial t} = k_1 A - k_2 M_1 - k_3 M_1 P_0 + D_1 \nabla^2 M_1$$

$$\frac{\partial M_2}{\partial t} = k_3 M_1 P_0 - k_4 M_2 - k_5 M_2 P_0 + D_2 \nabla^2 M_2$$

$$\frac{\partial P_0}{\partial t} = -k_3 M_1 P_0 - k_5 M_2 P_0$$

$$\frac{\partial P_1}{\partial t} = k_3 M_1 P_0$$

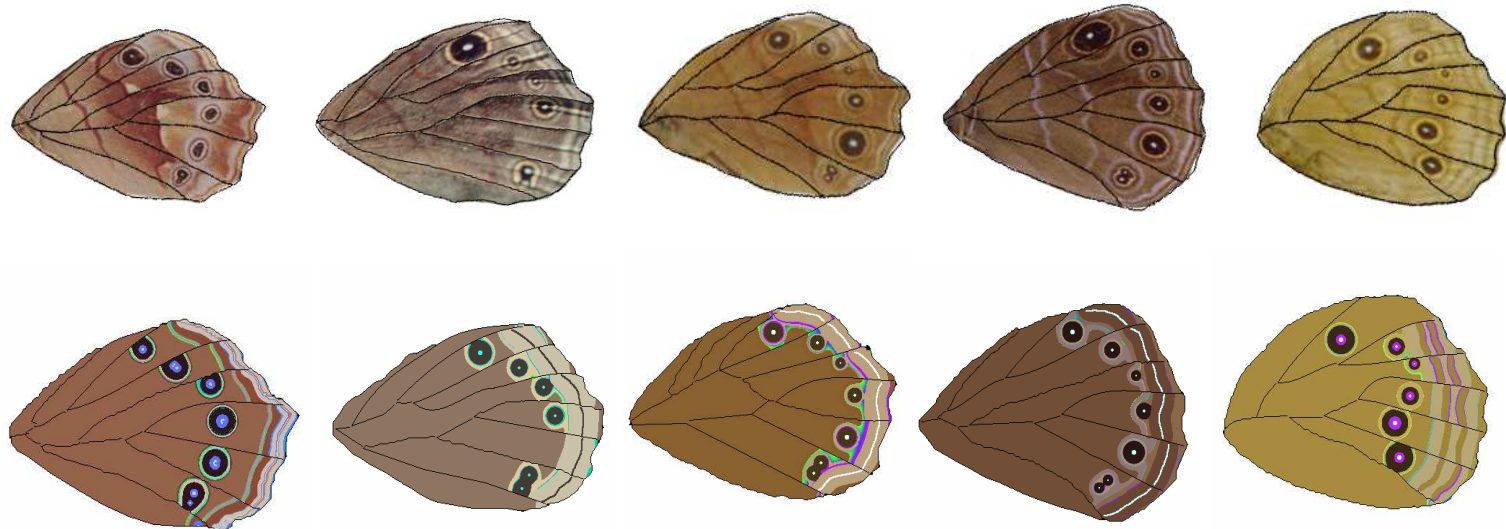
$$\frac{\partial P_2}{\partial t} = k_5 M_2 P_0$$

Dilão and Sainhas, 2003



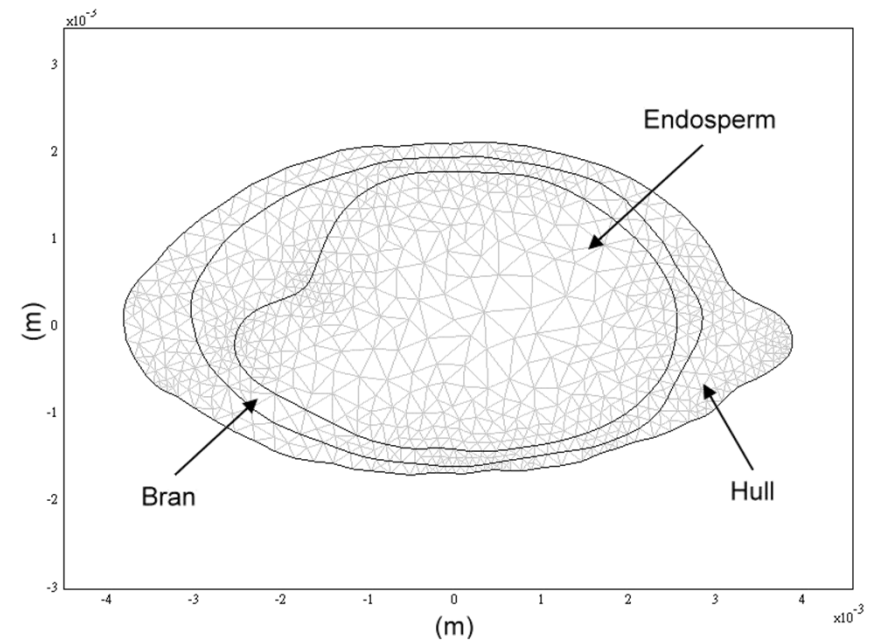
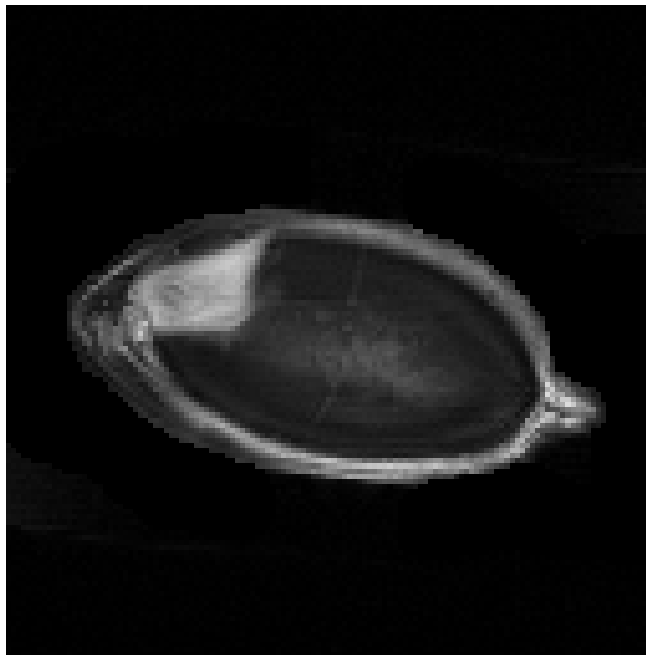
6.5 PDEs AND THE METHOD OF LINES

◎ Example Model of Butterfly Wing Patterns



6.5 PDEs AND THE METHOD OF LINES

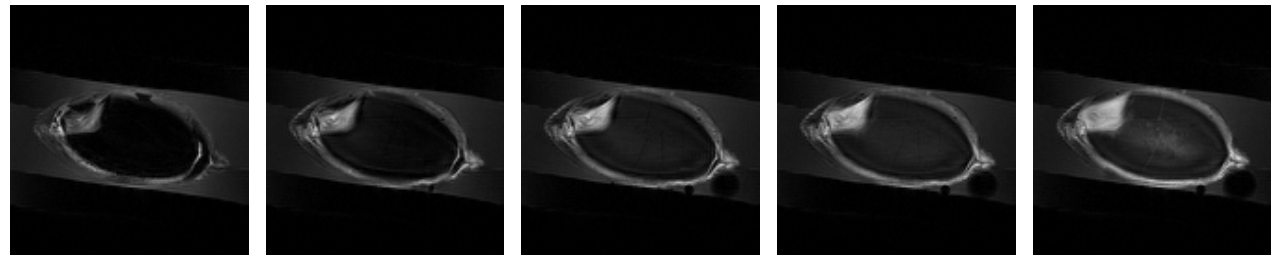
◎ Example Model of Rice Kernel Moisture Distribution



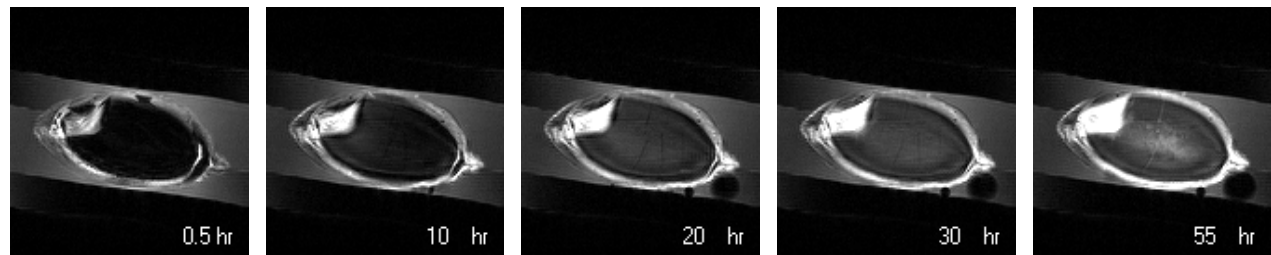
6.5 PDEs AND THE METHOD OF LINES

◎ Example Model of Rice Kernel Moisture Distribution

Original Images



Contrast Enhance



Simulated Results

